

Sezione 5/6 — Upgrade plan 0.1.30 → 0.1.40 + migrations

Cosa cambia, cosa va riscritto, in che ordine, come ci si torna indietro

5.1 Stato attuale → stato target

Elemento	Attuale	Target
Vendored package	ext_libs/s3dgraphy/ v0.1.30	ext_libs/s3dgraphy/ v0.1.40
System pip	0.1.15 (mascherato dal vendoring)	≥0.1.40 (allineato al vendored)
requirements.txt	s3dgraphy>=0.1.20	s3dgraphy>=0.1.40
Datamodel JSON	v1.5.2	v1.5.3 (canonical/reverse edge directionality)
Import in pyarchinit	from s3dgraphy import Graph, Node, Edge (3 simboli)	+ StratigraphicUnit, EpochNode, AuthorNode, GraphMerger, classification

BUONA NOTIZIA (AUDIT)

L'integrazione attuale usa **solo 3 simboli** di s3dgraphy: Graph , Node , Edge . Le API delle classi base sono compatibili dal 0.1.20 in poi. Il rischio di regressioni dirette è basso. La complessità sta nei *nuovi* moduli da adottare (GraphMerger, classification, aux_tracking).

5.2 Novità di 0.1.40 che useremo

Novità	Versione	Dove la usiamo
s3dgraphy.merge.GraphMerger	0.1.33	SyncEngine — merge 3-vie su pull e su 409 Conflict
s3dgraphy.merge.GraphMLPatcher	0.1.33	ParadataStore — patch incrementale del paradata.graphml
s3dgraphy.classification (get_family , is_real , iter_subtypes)	Unreleased→0.1.40	VocabProvider — alimenta i comboBox e il filtraggio per family
NegativeStratigraphicUnit (USN) + WorkingUnit (UL)	0.1.35 / Unreleased	GraphProjector — mapping per US negativi e WorkingUnits
aux_tracking (volatile vs bake save)	Unreleased	SyncEngine — distingue paradata "iniettati" da pyarchinit vs graph-native
importer.pyarchinit_importer	già esistente	GraphProjector — base della proiezione DB→graph
exporter.unified_xlsx_exporter	0.1.37	Bonus: export Excel del grafo come fallback offline

5.3 Audit del codice pyarchinit esistente

I 12 moduli in `modules/s3dgraphy/` usano una rappresentazione "string-tagged" (es. `node.node_type = 'stratigraphic_unit'`) invece dei tipi tipizzati di `s3dgraphy`. L'upgrade è anche occasione per allinearsi.

File	Modifiche	Effort
<code>s3dgraphy_integration.py</code> (1281 righe)	Refactor: <code>Node()</code> → <code>StratigraphicUnit()</code> , <code>EpochNode()</code> , ecc. tramite factory che legge il tipo da <code>VocabProvider</code> . Rimuovere setting di <code>node.node_type</code> stringa.	M (giorno)
<code>s3dgraphy_dot_bridge.py</code>	Aggiornare il visual mapping per leggere <code>family</code> via <code>classification.get_family()</code> invece di tabella hardcoded	S (3-4 ore)
<code>cidoc_crm_mapper.py</code>	Rimpiazzare <code>CRM_CLASSES</code> hardcoded con lookup in datamodel JSON via <code>VocabProvider</code>	S (2-3 ore)
<code>matrix_graph_visualizer.py</code> , <code>matrix_visualizer_qgis.py</code> , <code>plotly_visualizer.py</code>	Filter expressions <code>"node_type"</code> = <code>'stratigraphic_unit'</code> → <code>family = 'real'</code> tramite <code>classification</code> API	S (1-2 ore ciascuno)
<code>blender_integration.py</code>	Verifica solo: usa <code>node_type</code> come stringa, ok lasciarlo se l'export è verso Blender (formato proprio)	XS (review)
<code>graphml_spatial_enhancer.py</code> , <code>spatial_grouping_manager.py</code> , <code>simple_graph_visualizer.py</code>	Audit per verificare che non usino API rimosse. Probabile zero modifiche.	XS (audit)
<code>graphviz_visualizer.py</code>	Compatibile, niente da fare	—
<code>modules/utility/pyarchinit_i18n_stratigraphic.py</code>	Refactor in <code>VocabProvider</code> (vedi sezione 3). Legacy file mantenuto come adapter di compatibilità per 1 release.	M (giorno)

Stima complessiva: ~2-3 giorni di refactor + 1-2 giorni di test = 1 settimana persona.

5.4 Procedura di upgrade — step by step

1. **Branch checkpoint** — creare `feature/s3dgraphy-bridge-0.1.40` dal branch corrente. Tag prima del refactor: `pre-s3dgraphy-040` (rollback rapido).
2. **Test baseline** — eseguire la suite test attuale e salvare il risultato. Niente refactor finché non c'è una baseline verde.
3. **Replace vendored package:**

```
cd ext_libs/
rm -rf s3dgraphy s3dgraphy-0.1.30.dist-info
pip install s3dgraphy==0.1.40 --target . --no-deps
```

4. **Smoke test post-replace** — riavviare QGIS, aprire una scheda US, verificare che `S3DGRAPHY_AVAILABLE` sia True e che le matrici esistenti si renderizzino. Niente refactor ancora.
5. **Migrazione vocabolario USVA/USVB/USVC → USVs/USVn** — eseguire `scripts/migrations/2026_05_us_vocabulary_alignment.py --dry-run`, verificare il diff, poi `--apply`.
6. **Backfill UUIDs** — eseguire `scripts/migrations/2026_05_node_uuid_backfill.py` (aggiunge la colonna `node_uuid` e popola con UUID v7).
7. **Refactor uno-a-uno** dei file in `modules/s3dgraphy/` (ordine consigliato): `cidoc_crm_mapper` → `s3dgraphy_integration` → `matrix_visualizer_qgis` → resto. Test dopo ogni file.
8. **Implementazione nuovi moduli** — VocabProvider, GraphProjector, ParadataStore, SyncEngine, DatacenterClient, ConflictResolver in `modules/s3dgraphy/sync/`.
9. **Update requirements.txt** — bump a `s3dgraphy>=0.1.40`.
10. **Update CHANGELOG + bump versione plugin** — `5.0.25-alpha` → `5.1.0-alpha` (minor, perché aggiunge feature di sync; il refactor interno non è breaking per l'utente).

5.5 Migrazioni dati richieste

Script	Cosa fa	Idempotente?
<code>2026_05_us_vocabulary_alignment.py</code>	USVA/USVB → USVs, USVC → USVn nelle tabelle <code>us_table</code> di tutti i database aperti. Logga su <code>migration.log</code> .	Sì (re-run no-op)
<code>2026_05_node_uuid_backfill.py</code>	<code>ALTER TABLE</code> per aggiungere <code>node_uuid</code> TEXT a 4 tabelle, poi <code>UPDATE</code> con UUID v7 dove NULL. Crea index unique.	Sì (verifica colonna esistente)
<code>2026_05_paradata_bootstrap.py</code>	Per ogni progetto QGIS, crea <code>paradata.graphml</code> vuoto se non esiste. Popola con AuthorNode dalle preferenze utente.	Sì (skip se già presente)

Tutti gli script supportano `--dry-run`, `--apply`, `--rollback`. Sono accessibili da *menu QGIS → PyArchInit → Manutenzione → Migrazioni*.

5.6 Strategia di rollback

Punto di fallimento	Rollback
Replace ext_libs fallisce	<code>git checkout ext_libs/s3dgraphy</code> (era versionato)
Smoke test rosso post-replace	Stesso comando, più <code>git tag pre-s3dgraphy-040</code>
Migrazione vocabolario corrompe DB	Lo script salva backup automatico in <code><db>.pre_alignment_<date></code> ; rollback è <code>cp</code>
UUID backfill su PostgreSQL fallisce parzialmente	<code>ALTER TABLE ... DROP COLUMN node_uuid CASCADE</code> (script <code>--rollback</code> lo fa)
Refactor di un singolo modulo introduce regressione	Commit per modulo → <code>git revert <sha></code> selettivo
Sync engine instabile in produzione	Feature flag <code>QSettings("pyarchinit/sync_enabled", False)</code> disabilita push/pull, lasciando funzionante l'export read-only

5.7 Test richiesti per dichiarare l'upgrade pronto

- **Smoke:** import `s3dgraphy` ok, `S3DGRAPHY_AVAILABLE = True` , versione = 0.1.40
- **Backward compat:** aprire un progetto `pyarchinit` pre-upgrade (senza `node_uuid`) → niente crash, prompt di migrazione
- **Round-trip:** DB → `GraphProjector` → `ParadataStore.merge` → export `GraphML` → re-import → diff zero
- **Vocabulary:** aggiungere un tipo nel JSON → `comboBox` lo mostra senza riavviare QGIS
- **Sync E2E:** con datacenter mock (HTTP server locale), push → modifica remota → pull → conflitto → resolution → push final
- **Multi-DB:** ripetere su SQLite, PostgreSQL, Spatialite
- **Migrazione vocabolario:** DB con USVA/USVB/USVC viene allineato senza perdita dati, query post-migrazione restituiscono USVs/USVn